

Problem Set (Tugas Mandiri) – Minggu 3

Proyek Sistem Operasi: IPC, Pipes & Redirection

Novalio Daratha
Program Studi Informatika
Universitas Bengkulu

2026

Petunjuk Tugas Mandiri

Problem Set ini dikerjakan secara mandiri sebagai pendalaman materi di luar kelas. Soal mencakup seluruh ranah kognitif C1–C6. Kumpulkan jawaban dalam format PDF beserta kode sumber pendukung ke repositori GitLab kelas sebelum pertemuan Minggu 4.

Daftar Soal

Soal 1. [C1 – Mengingat] (Bobot: 10%)

Sebutkan nilai konstanta POSIX (definisi makro) untuk *file descriptor* standar berikut:

- a. *Standard Input* (stdin)
- b. *Standard Output* (stdout)
- c. *Standard Error* (stderr)

Soal 2. [C2 – Memahami] (Bobot: 15%)

Jelaskan apa yang dimaksud dengan sinyal SIGPIPE pada sistem operasi Linux. Kapan sistem operasi akan mengirimkan sinyal ini ke suatu proses yang sedang menulis ke *pipe*?

Soal 3. [C3 – Menerapkan] (Bobot: 20%)

Tuliskan program C lengkap di mana sebuah *Parent Process* mengirimkan sebuah array berisi 10 bilangan bulat acak ke *Child Process* melalui *pipe*. *Child Process* harus menghitung nilai rata-rata dari array tersebut dan mencetaknya ke terminal.

Soal 4. [C4 – Menganalisis] (Bobot: 20%)

Perhatikan potongan kode berikut:

```
int fd[2];
pipe(fd);
if (fork() == 0) {
    char ch = 'A';
    write(fd[1], &ch, 1);
} else {
    wait(NULL);
    char res;
    read(fd[0], &res, 1);
    printf("Hasil: %c\n", res);
}
```

Analisislah: Mengapa penempatan `wait(NULL)` sebelum `read()` pada kode di atas dapat berpotensi memicu *Deadlock* jika ukuran data yang ditulis oleh proses anak melebihi kapasitas buffer *pipe* kernel (biasanya 64 KB)?

Soal 5. [C5 – Mengevaluasi] (Bobot: 15%)

Bandingkan fitur pengalihan *overwrite* (>) dan *append* (>>) pada UNIX shell. Evaluasi parameter *flags* apa saja yang harus dioperasikan pada fungsi `open()` untuk merepresentasikan masing-masing mode tersebut secara presisi.

Soal 6. [C6 – Mencipta] (Bobot: 20%)

Rancanglah sebuah fungsi C modular bernama `int execute_redirection(char *argv[], char *infile, char *outfile, int append_mode)` yang mampu menangani pengalihan input dan output sekaligus secara aman, serta mengembalikan kode status eksekusi perintah.